

GoTrip – KI-gestützte Gruppenreiseplanung

Modul: Mensch-Computer-Interaktion (HCI) – SoSe 2026, Meilenstein 3 (Implementierung)

Hochschule: h_da – Hochschule Darmstadt

Team: Mohamad Haj Ahmad · Wajdy Eleyan · Amal Najah · Eya Mathlouthi

Live-Demo: <https://gotrip-9id7.onrender.com>

Über GoTrip

GoTrip ist eine mobile Web-App für die gemeinsame Gruppenreiseplanung. Gruppen koordinieren Verfügbarkeiten, Budgets und Interessen – die App berechnet daraus automatisch einen KI-Score für jeden Reisevorschlag, basierend auf echten Klimadaten, Biodiversitätsdaten und NASA-Satellitendaten.

Kernidee

KI-Score (40%) + Gruppen-Sternebewertung (60%) = Hybrid-Score

Die App vergleicht das vorgeschlagene Reiseziel mit den Präferenzen aller Mitglieder. Wer Strand wählt und Frankfurt vorschlägt, bekommt einen niedrigen Score – weil Frankfurt im Binnenland liegt (echte Küstendaten via Open-Meteo Marine API).

Tech-Stack

Schicht	Technologie	Version
Frontend	React + TypeScript + Vite	React 19 , Vite 8
Styling	Tailwind CSS	v4
Routing	React Router	v7
Formulare	React Hook Form + Zod	–
Icons	lucide-react	–
State	React Context API	–
Persistenz (lokal)	localStorage	–
Persistenz (server)	PostgreSQL (Neon Cloud)	–
Backend	Express.js	–
Deployment	Render.com	–
Echtzeit-Sync	HTTP-Polling (6 s)	–
Klimadaten	Copernicus ERA5 via Open-Meteo	kostenlos, kein Key
Biodiversität	GBIF Occurrence API	kostenlos, kein Key
Wetterdaten	NASA POWER API	kostenlos, kein Key

Küstenerkennung	Open-Meteo Marine API	kostenlos, kein Key
Satellitendaten	NASA Earthdata	Token erforderlich
Geocoding	Open-Meteo Geocoding	kostenlos, kein Key

Funktionsübersicht

Screen	Funktion
Login	Name eingeben, kein Account nötig
Home	Alle eigenen Reisen auf einen Blick
Trip erstellen	Name, Zeitraum (iOS-Wheel-Picker), Budget, Gruppenname
Freunde einladen	Link-Share via WhatsApp / Web-Share-API
Gruppenmitglieder	Mitglieder sehen, Admin-Rolle
Verfügbarkeit	Kalender: Verfügbar / Vielleicht / Nicht verfügbar
Präferenzen	Budget-Picker + 9 Interessen-Chips (Strand, Stadt, Natur ...)
KI-Empfehlung	Reiseziele vorschlagen, analysieren lassen und bewerten
Übersicht	Schlussfolgerung + Gruppen-Status + Pro-Mitglied-Details
Aktivitäten	Aktivitäten vorschlagen + Thumbs-up-Voting
Budget-Tracker	Ausgaben erfassen und pro Mitglied aufschlüsseln
Reiseplan	Itinerary mit Zeitslots für jeden Tag

Installation & lokaler Start

Voraussetzungen

- Node.js ≥ 18
- npm ≥ 9

1. Repository klonen

```
git clone https://github.com/mo518525/Trip-Planer.git
cd Trip-Planer
```

2. Frontend starten

```
npm install
npm run dev
```

Browser öffnet sich auf <http://localhost:5173>

3. Backend starten (für Datenbanksynch + NASA Earthdata)

```
cd server
npm install
node index.js
```

Der Server läuft auf <http://localhost:3001>

Ohne Backend: Die App funktioniert vollständig. Klimadaten, Biodiversität und NASA POWER werden direkt im Browser abgerufen. Nur die Gruppensynchronisation via PostgreSQL und NASA Earthdata benötigen den Server.

4. Umgebungsvariablen (Server)

Datei `server/.env` anlegen:

```
DATABASE_URL=postgresql://... # Neon PostgreSQL Connection String
EARTHDATA_TOKEN=...           # NASA Earthdata Bearer Token (optional)
```

`server/.env` ist in `.gitignore` – wird **nicht** ins Repository committed.

5. Produktions-Build

```
npm run build
# Build liegt in dist/
```

Architektur

Ordnerstruktur

```
gotrip/
├─ src/
│  ├─ components/
│  │  ├─ availability/ # AvailabilityCalendar, GroupHeatmap
│  │  ├─ calendar/    # CalendarDay, MonthGrid
│  │  ├─ members/     # MemberCard, InviteLink
│  │  ├─ preferences/ # InterestChips, BudgetSlider
│  │  ├─ recommendations/ # DestinationCard, ScoreRing, DataSourceBadges
│  │  ├─ shared/       # PageHeader, BottomNav, TripScreen, Toast
│  │  ├─ sheets/       # AvailabilitySheet, RecommendationSheet, ...
│  │  ├─ trips/        # TripCard, TripForm
│  │  ├─ ui/           # Button, Input, WheelDatePicker
│  │  └─ voting/       # StarRating
│  └─ context/
│     ├─ AuthContext.tsx # Authentifizierung (localStorage)
│     ├─ TripContext.tsx # Trip-CRUD + State
│     └─ LanguageContext.tsx # i18n (DE / EN / ES)
│  └─ i18n/
│     └─ translations.ts # Alle UI-Texte in 3 Sprachen
│  └─ pages/             # 16 Screens (je eine Route)
```

```

|   ├── services/
|   |   ├── llmService.ts      # Koordiniert alle API-Calls
|   |   ├── realData.ts       # Copernicus, GBIF, NASA POWER, Geocoding
|   |   ├── scoreEngine.ts    # Regelbasierte Score-Berechnung
|   |   └── tripSync.ts       # Echtzeit-Sync + Benachrichtigungen
|   ├── types/                # TypeScript-Interfaces
|   └── utils/                 # storage.ts, scoring.ts, destinationImage.ts
├── server/
|   ├── index.js              # Express: DB-Sync + NASA Earthdata Proxy
|   └── package.json
├── render.yaml               # Render.com Deployment-Konfiguration
└── README.md

```

Datenfluss

```

Nutzer-Aktion
  |
  ▼
React Component → localStorage (sofort, offline-fähig)
  |
  ▼
scheduleSync() → POST /api/trip → Neon PostgreSQL
  |
  ▼
pullTrip() alle 6 s ← Andere Mitglieder empfangen Änderungen
  |
  ▼
gotrip-notification Event → Toast-Benachrichtigung (überall sichtbar)

```

KI-Score-Berechnung

Die Logik liegt in `src/services/scoreEngine.ts` und `src/services/realData.ts` :

Gesamtscore = Klimakomfort × 0,4 + Interessenerfüllung × 0,6

Klimakomfort (aus echten ERA5-Daten):

Temperatur 20–26 °C → +25 Punkte

Niederschlag < 20 mm → +12 Punkte

Sonne ≥ 9 h/Tag → +8 Punkte

Interessenerfüllung (aus Geocoding + Marine API):

beach → !coastal ? 8 : (warm ? 95 : 65)

city → pop ≥ 500.000 ? 92 : 18

nature → species > 1.000.000 ? 85 : 55

Der Score wird **dynamisch** bei jeder Anzeige mit den aktuellen Gruppenpräferenzen neu berechnet – keine veralteten gecachten Werte.

Mehrsprachigkeit

Die App unterstützt **Deutsch, Englisch und Spanisch**. Alle UI-Texte sind in `src/i18n/translations.ts` zentral verwaltet.

Deployment (Render.com)

- Frontend-Build (`dist/`) wird vom Express-Server ausgeliefert
- Ein einziger Service für Frontend + Backend
- Automatisches Re-Deployment bei jedem Push auf `main`
- Umgebungsvariablen `DATABASE_URL` und `EARTHDATA_TOKEN` im Render-Dashboard

Externe APIs (alle kostenlos)

API	Verwendung
Open-Meteo Archive (ERA5)	Temperatur, Niederschlag, Sonnenstunden
Open-Meteo Marine	Küstenerkennung
Open-Meteo Geocoding	Koordinaten, Einwohnerzahl
GBIF Occurrence	Artenvielfalt
NASA POWER	Sonneneinstrahlung, Luftfeuchte, Wind
NASA Earthdata	Satellitenaufnahmen (optional, Token)
Neon PostgreSQL	Gruppendaten-Sync (Cloud-Datenbank)

Autoren

Bereich	Autoren
Frontend-Entwicklung (alle Seiten)	Mohamad Haj Ahmad, Wajdy Eleyan, Amal Najah, Eya Mathlouthi
Backend, Datenbank, API-Integration, Score-Engine	Mohamad Haj Ahmad, Eya Mathlouthi, Wajdy Eleyan
Sync-System, Echtzeit-Kommunikation	Amal Najah, Wajdy Eleyan
Design & UI-Konzept	Amal Najah, Eya Mathlouthi
Dokumentation (MS3 & MS4)	Amal Najah, Eya Mathlouthi
Projektmanagement	Mohamad Haj Ahmad, Wajdy Eleyan

Bekannte Einschränkungen

- Authentifizierung ist Mock-basiert (kein echtes User-Management mit Passwort)
- Instagram-Share öffnet nur einen Hinweisdiallog (Instagram hat keine Web-Share-API)
- Render.com Free Tier: Server schläft nach 15 min Inaktivität (Cold Start ~30 s)

Release Notes

Siehe [RELEASE NOTES.md](#)